

Disproof of conjecture 00000001003

earthking11

2026-09-14

Verdict: FALSE

Conjecture 00000001003 is false. We disprove it at the very first odd prime power, $q = 3$: the conjecture predicts a maximal cap size of

$$q^2 + q + 1 + \lfloor (q+1)/3 \rfloor \Big|_{q=3} = 9 + 3 + 1 + \lfloor 4/3 \rfloor = 14,$$

but $\text{PG}(4, 3)$ contains an explicit cap of 20 points, and $20 > 14$. The failure is not an artefact of $q = 3$: the same formula is beaten in $\text{PG}(4, 5)$ (an explicit 47-point cap against the predicted 33), the even- q clause fails at $q = 2$ (a 16-point cap against the predicted 10), and the description “attained by an elliptic quadric with an explicit three-point augmentation” is internally inconsistent with the stated formula. The refutation is unconditional; no unproved hypothesis is used.

1 The conjecture, quoted verbatim

From `conjectures/00000001003.md`:

English. Definition: A cap is a point set of $\text{PG}(n, q)$ with no three collinear. Conjecture: The maximal cap size in $\text{PG}(4, q)$ is $q^2 + q + 1 + \lfloor (q+1)/3 \rfloor$ for all odd q , attained by an elliptic quadric with an explicit three-point augmentation; for even q the value increases by 2.

The file is bilingual; the Chinese original is reproduced verbatim in `README.md` (this PDF does not embed CJK glyphs). The two versions are identical in content.

Definition 1 (cap). A *cap* in $\text{PG}(n, q)$ is a set of points no three of which are collinear. Its *size* is its cardinality; it is *inclusion-maximal* if no further point can be added while remaining a cap.

Throughout, “maximal” is read as *maximum* (the largest size), which is the only reading under which the conjecture states a numerical value. We also test the inclusion-maximal reading in Section 7.

2 The explicit 20-point cap in $\text{PG}(4, 3)$

We use homogeneous coordinates on $\text{PG}(4, 3)$; every point is written in normalised form, i.e. its first nonzero coordinate is 1. The following 20 points form a cap:

(1, 0, 0, 0, 2)	(0, 0, 0, 1, 2)	(1, 2, 2, 1, 1)	(0, 1, 0, 0, 0)	(1, 0, 2, 1, 2)
(1, 0, 1, 2, 0)	(0, 1, 0, 1, 0)	(1, 2, 1, 2, 1)	(1, 1, 2, 2, 0)	(1, 2, 0, 1, 2)
(1, 2, 0, 1, 0)	(1, 2, 1, 0, 2)	(1, 0, 1, 1, 0)	(0, 1, 1, 1, 2)	(0, 0, 1, 0, 0)
(0, 1, 2, 2, 0)	(1, 0, 2, 2, 2)	(1, 2, 1, 2, 2)	(0, 1, 2, 2, 2)	(1, 1, 0, 1, 1)

That is, no three of these 20 points are collinear. Since the conjecture predicts 14 at $q = 3$, this already gives

$$M_2(5, 3) \geq 20 > 14 = q^2 + q + 1 + \lfloor (q+1)/3 \rfloor \big|_{q=3},$$

so the $q = 3$ instance of the quantified claim “for all odd q ” is false.

2.1 Three independent verifications

All three were carried out and agree on the same answer, 0 collinear triples. The script `reproduce.py` performs the first two from scratch.

1. **Line membership.** $\text{PG}(4, 3)$ has 121 points and 1210 lines; each line has $q + 1 = 4$ points. Enumerating all lines and intersecting them with the $\binom{20}{3} = 1140$ triples gives 0 collinear triples.
2. **Rank over $\mathbb{F}_3$.** Three projective points are collinear iff the rank of their three coordinate vectors is ≤ 2 . Gaussian elimination over $\mathbb{F}_3$ on all 1140 triples gives 0 triples of rank ≤ 2 .
3. **Kernel-checked Lean.** The executable checker in `lean4/Main.lean` (core Lean, no Mathlib, no sorry) returns `true` on the list; the theorem `pts20_isCap` is proved by `decide` and depends on no axioms.

3 The exact value 20 (SAT-certified upper bound)

We have proved $M_2(5, 3) \geq 20$. The matching upper bound $M_2(5, 3) \leq 20$ (i.e. no 21-point cap exists) is a finite computation, which we report as *SAT-certified* rather than kernel-checked. The argument is:

Proposition 2 (upper bound, SAT certificate). $M_2(5, 3) \leq 20$.

Argument. Let $C \subseteq \text{PG}(4, 3)$ be a cap with $|C| \geq 21$. Any set of ≥ 21 points in $\text{PG}(4, 3)$ spans $\text{PG}(4, 3)$, hence contains 5 linearly independent points. The group $\text{GL}(5, 3)$ acts transitively on ordered bases of $\mathbb{F}_3^5$, so after a projectivity we may assume that the 5 standard basis points (a partial frame) lie in C . The remaining points are then searched subject to the constraint that no three chosen points are collinear; with the frame fixed and $|C| \geq 21$ required, the resulting CNF is UNSAT (checked with a SAT solver using a cardinality encoding). Therefore no such C exists. $\square$

This is a computer-assisted certificate over a finite search space, not a kernel-checked proof; it is recorded here for completeness. The disproof of the conjecture does *not* depend on it: the lower bound $M_2(5, 3) \geq 20 > 14$ already refutes the value 14. The value 20 agrees with the classical table entry $M_2(5, 3) = 20$.

4 The general formula fails at $q = 5$

The conjecture quantifies over all odd q , so failing at $q = 3$ already disproves it. It is nevertheless false at the next odd prime power as well, and by a wide margin. In $\text{PG}(4, 5)$ (781 points) an

explicit cap of 47 points exists; the conjecture predicts

$$25 + 5 + 1 + \lfloor 6/3 \rfloor = 33.$$

The 47 points are exhibited explicitly in `reproduce.py` and verified there by two methods (pairwise line membership and rank over $\mathbb{F}_5$): all $\binom{47}{3} = 16215$ triples have rank 3, so the cap is valid. Hence $M_2(5, 5) \geq 47 > 33$. (A cap of size 46 at $q = 5$ was found earlier; the deterministic list shipped in `reproduce.py` has 47 points and is strictly stronger.)

5 The even- q clause fails at $q = 2$

For even q the conjecture adds 2 to the formula, giving at $q = 2$

$$4 + 2 + 1 + \lfloor 3/3 \rfloor + 2 = 10.$$

But $\text{PG}(4, 2)$ (31 points) contains the 16-point cap

$$\{ (1, v) : v \in \mathbb{F}_2^4 \},$$

and in fact $M_2(4, 2) = 16 = 2^4$, the classical value $M_2(n, 2) = 2^n$. The lower bound 16 is verified in `reproduce.py`; the upper bound $M_2(4, 2) \leq 16$ is again a finite SAT certificate (≥ 17 is UNSAT), reported as such. Thus the even- q clause is off by 6 at $q = 2$: not merely does the formula plus 2 fail, it fails in the opposite direction (the true value is larger, not smaller).

6 Internal inconsistency of the description

The conjecture says the value is “attained by an elliptic quadric with an explicit three-point augmentation”. An elliptic quadric in this setting contributes $q^2 + 1$ points, and augmenting it by three points gives

$$q^2 + 4.$$

For $q = 3$ this is 13, whereas the stated formula gives 14; in general

$$q^2 + q + 1 + \lfloor (q + 1)/3 \rfloor - (q^2 + 4) = q - 3 + \lfloor (q + 1)/3 \rfloor > 0 \quad (q \geq 3 \text{ odd}).$$

So the construction does not attain the stated value (and the stated value is not produced by the construction). The two halves of the sentence are inconsistent with each other. The same mismatch occurs at $q = 5$: 29 from the quadric description versus 33 from the formula.

7 Alternative readings

We tried to rescue the conjecture by varying the reading; none succeeds.

Reading	Outcome
Relax “cap” (allow some collinear triples)	Contradicts the file’s own definition of a cap and has no canonical value; the conjecture then states no definite number, so it is not a well-posed claim.
“Maximal” means inclusion-maximal	The 20-point cap above is inclusion-maximal (verified: no point of $\text{PG}(4, 3)$ can be added to it). A survey of 3000 random greedy inclusion-maximal caps produced sizes 16–20 (distribution 16 : 84, 17 : 292, 18 : 1889, 19 : 459, 20 : 276), <i>never</i> 14. So this reading does not force the value 14 either.
“The formula is meant for $q \geq 5$ ”	Fails at $q = 5$: an explicit 47-point cap versus the predicted 33 (Section 5).
Even- q clause taken as the base formula	Fails at $q = 2$: 16 versus 10 (Section 6).

The first line is the only one that changes the meaning of a term rather than a quantifier range; under the file’s own definition of “cap” it does not apply.

8 Reproducibility

`reproduce.py` (Python 3 standard library only, no third-party packages) rebuilds $\text{PG}(4, 3)$, checks 121 points and 1210 lines, verifies the 20 points are distinct normalised projective points, and checks all $\binom{20}{3} = 1140$ triples by *both* line membership and rank over $\mathbb{F}_3$, obtaining 0 collinear triples in each case; it also verifies $20 > 14$, that the 20-cap is inclusion-maximal, the 47-point cap in $\text{PG}(4, 5)$, the 16-point cap in $\text{PG}(4, 2)$, and the quadric/formula mismatch. It prints **PASS** and exits 0 exactly when every check holds:

```
$ python3 reproduce.py
... PASS: all checks verified; conjecture 00000001003 is FALSE.
```

9 Formalisation

The refutation at $q = 3$ is formalised in the Lean 4 project under `lean4/` (core Lean only, `import Std`, no `Mathlib`, no `sorry`, no `native_decide`). Points are a structure `Pt` with five `Nat` fields, arithmetic is reduced with `% 3`, and collinearity is decided by the executable predicate `bad`, which is complete over $\mathbb{F}_3$: the projective line through two distinct points p, q is $\{p, q, \text{normalize}(p + q), \text{normalize}(p + 2q)\}$. The main statements are

```
def formula (q : Nat) : Nat := q*q + q + 1 + (q+1)/3
theorem pts20_isCap : isCap pts20 = true
theorem formula_three : formula 3 = 14
theorem refutes_conjecture :
  ∃ S : List Pt, S.length = 20 ∧ isCap S = true ∧ formula 3 < S.length
```

All eleven theorems are proved by `decide` and the audit (`lake env lean Check.lean`, using `#print axioms`) reports “does not depend on any axioms” for every one of them — in particular no `sorryAx` and no `Lean.ofReduceBool`. See `lean4/README.md` for the statement table, proof strategy, and scope note.

Status against the submission rules

Rule 3 requires a LaTeX source, a PDF document, and a Lean 4 project.

- **LaTeX source** — `main.tex`, a standalone article using `geometry`, `amsmath`, `amssymb`, `amsthm`, `array`, `parskip`, `booktabs`; compiles with `tectonic --outdir build main.tex`.
- **PDF** — at `build/main.pdf`.
- **Lean 4 project** — `lean4/` with `lean-toolchain` pinned to `leanprover/lean4:v4.33.1`, `lakefile.toml` (library `Main`, name `tlmc1003`, no dependencies), `Main.lean`, `Check.lean` (`#print axioms`), and `lean4/README.md`. `lake build` exits 0 and the audit reports no sorryAx.